

LEARN PYTHON PROGRAMMING – BEGINNER TO MASTER

Section: 9. Regular Expression

Lecture: 96. Regular Expressions

Section 1: Lecture Summary

This lecture introduces the foundational concepts of regular expressions (regex) and their uses, particularly in Python programming. It starts by explaining why regex is important in areas like data science, natural language processing, and compiler construction. The lecture outlines the primary applications of regex: text processing (searching and replacing text with patterns), data validation (ensuring data formats like emails are correct), and extracting information from large text sources. The key concept of pattern matching is introduced with simple examples of patterns and their matches against strings. The lecture then explains Python's `re` module, which provides functions for implementing regex operations. Several functions are introduced, including `compile()`, `match()`, `fullmatch()`, `search()`, and `findall()`, with explanations of their purposes, such as checking for matches at the start of strings, validating entire strings, searching anywhere within strings, and finding all occurrences. The lecture concludes with practical demonstrations of these functions in a Jupyter notebook environment, illustrating how to perform regex operations in Python.

Section 2: Key Concepts and Explanations

- Regular Expressions (Regex):
 - Definition: Patterns describing sets of strings used for pattern matching.
 - Origins: Rooted in mathematics, widely supported in programming languages.
 - Uses:
 - Text Processing: Searching and replacing text based on patterns.
 - Data Validation: Checking if data matches specified formats (e.g., email validation).
 - Information Extraction: Retrieving structured data from large blocks of text.

- Pattern Matching:

- Pattern: A string or sequence representing a search criterion.

- Matching Modes:

- Match at the beginning of a string.

- Match anywhere within a string.

- Find all occurrences.

- Exact full string match.

- Examples:

- Pattern `"and"` matches "android", "handle", and "and".

- Pattern `"com|org"` matches either "com" or "org".

- Pattern `[abcd]` matches any single character `a`, `b`, `c`, or `d`.

- Pattern `[abc]+` matches one or more repetitions of `a`, `b`, or `c` (e.g., "a", "aa", "abc", "bb").

- Python `re` Module:

- Module: A library containing pre-built regex functions, must be imported (`import re`).

- Import Variants:

- `import re`

- `import re as regex` (aliasing)

- `from re import *` (import everything directly)

- Key Functions:

- `compile(pattern, flags=0)`: Compiles a pattern into a regex object for reuse.

- `match(pattern, string, flags=0)`: Checks if pattern matches at the start of string.

- `fullmatch(pattern, string, flags=0)`: Checks if entire string matches the pattern.

- `search(pattern, string, flags=0)`: Searches for the first occurrence of the pattern anywhere in string.

- `findall(pattern, string, flags=0)`: Returns all non-overlapping matches of pattern in string as a list.
- `split(pattern, string, maxsplit=0, flags=0)`: Splits string by occurrences of pattern.
- Regex Match Objects:
 - Returned by functions like `match()` and `search()` when successful.
 - Can extract matched text via `.group()`.
 - Span (start and end position) of match accessible by `.span()`.

Section 3: Example Code and Use Cases

Example 1: `match()` - Match pattern at start of string

```
import re

pattern = 'abc'
string = 'abcdef'
result = re.match(pattern, string)
print(result) # Output: <re.Match object; span=(0, 3), match='abc'>
```

Explanation: Checks if 'abc' matches the start of 'abcdef'. Returns a match object.

Example 2: `fullmatch()` - Full string match

```
pattern = 'python'
string = 'python'
result = re.fullmatch(pattern, string)
print(result.group()) # Output: python
```

Explanation: Checks if entire string 'python' matches pattern 'python'.

Example 3: `search()` - Find first occurrence anywhere

```
pattern = 'very'
string = 'Python is very easy'
result = re.search(pattern, string)
```

```
print(result.span()) # Output: (10, 14)
print(result.group()) # Output: very
```

Explanation: Finds 'very' in string at indices 10 to 14.

Example 4: findall() - Find all occurrences

```
pattern = 'can'
string = 'can you can a can as a canner?'
matches = re.findall(pattern, string)
print(matches) # Output: ['can', 'can', 'can', 'can']
```

Explanation: Finds all substrings matching 'can' and returns them as a list.

Section 4: Key Takeaways

- Regular expressions provide a powerful way to define text patterns and perform pattern matching, useful in text processing, data validation, and information extraction.
- Patterns can represent single strings, alternatives (`com|org`), character groups (`[abcd]`), or repetitions with quantifiers (`[abc]+`).
- Python's `re` module contains essential functions for regex: `compile()`, `match()`, `fullmatch()`, `search()`, and `findall()`.
- `match()` checks only at the beginning of a string, while `fullmatch()` checks the entire string strictly.
- `search()` locates the first match anywhere in the string; `findall()` returns all matches.
- Regex results are returned as match objects from which matched substrings and their positions can be extracted.
- Importing and using the `re` module correctly is necessary to utilize regex features in Python.
- Writing and understanding complex regex patterns requires studying quantifiers, special characters, and escape sequences, which are covered in subsequent lectures.

Lecture in Slides

Please Note: This document presents a curated selection of slides from the lecture; others have been excluded for conciseness.





Regular Expressions



Regular Expressions

Pattern Matching

Regular Expressions

Pattern Matching

Regular Expressions

Pattern Matching

Uses of Regular Expression

Regular Expressions

Pattern Matching

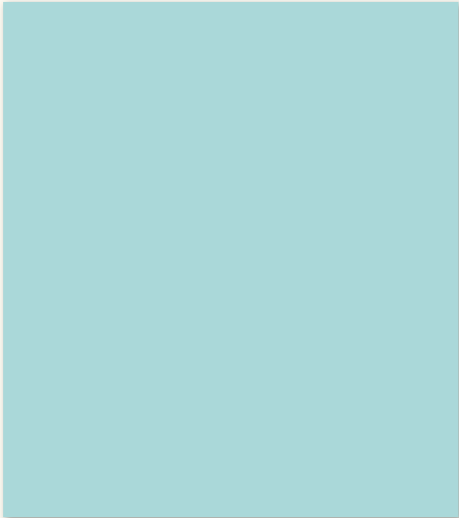
Uses of Regular Expression

Regular Expressions

Pattern Matching

Uses of Regular Expression

Text Processing



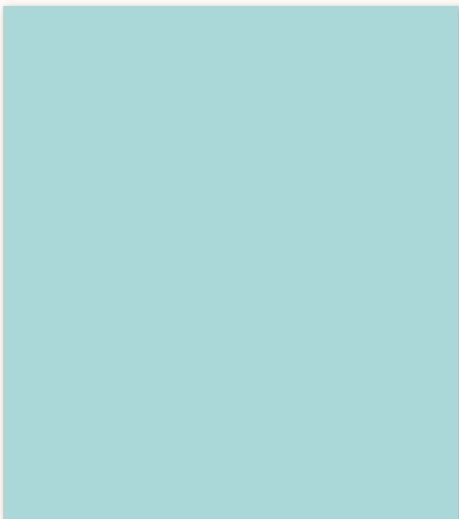
Regular Expressions

Pattern Matching

Uses of Regular Expression

Text Processing

Search and replace text



Regular Expressions

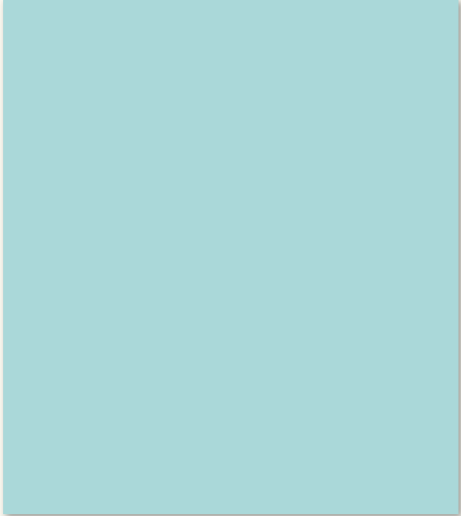
Pattern Matching

Uses of Regular Expression

Text Processing

Search and replace text





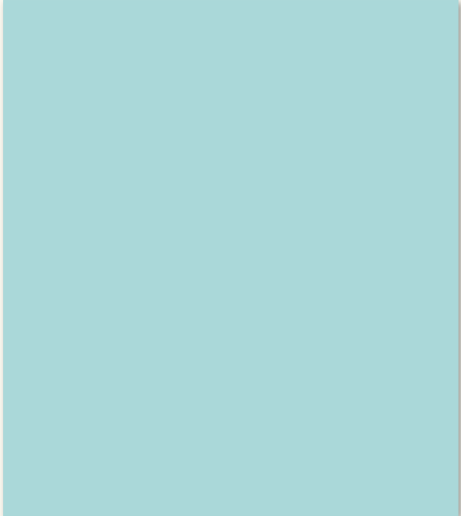
Regular Expressions

Pattern Matching

Uses of Regular Expression

Text Processing
Search and replace text

Data Validation



Regular Expressions

Pattern Matching

Uses of Regular Expression

Text Processing
Search and replace text

Data Validation
Check data is in proper format

Regular Expressions

Pattern Matching

Uses of Regular Expression

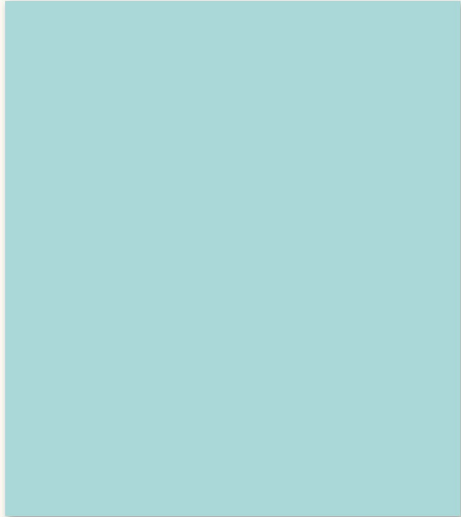
Text Processing

Search and replace text

Data Validation

Check data is in proper format





Regular Expressions

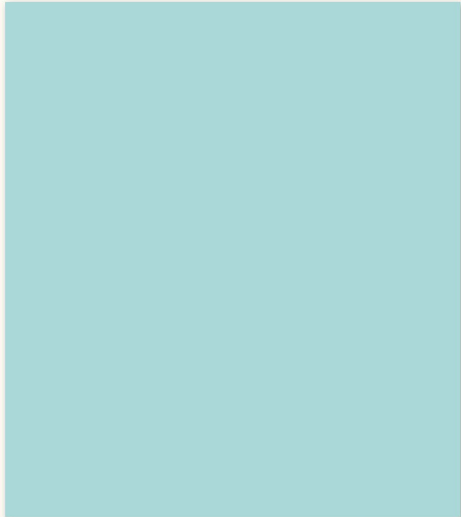
Pattern Matching

Uses of Regular Expression

Text Processing
Search and replace text

Data Validation
Check data is in proper format

Extracting Information



Regular Expressions

Pattern Matching

Uses of Regular Expression

Text Processing
Search and replace text

Data Validation
Check data is in proper format

Extracting Information
Extracting info from large text

Regular Expressions

Pattern Matching

Uses of Regular Expression

Text Processing

Search and replace text

Data Validation

Check data is in proper format

Extracting Information

Extracting info from large text



Regular Expressions

Pattern Matching

Topics

Regular Expressions

Pattern Matching

Topics

What is Pattern Matching?

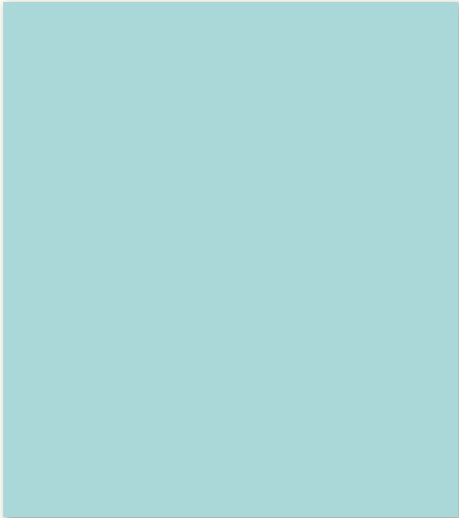
Regular Expressions

Pattern Matching

Topics

What is Pattern Matching?

Functions



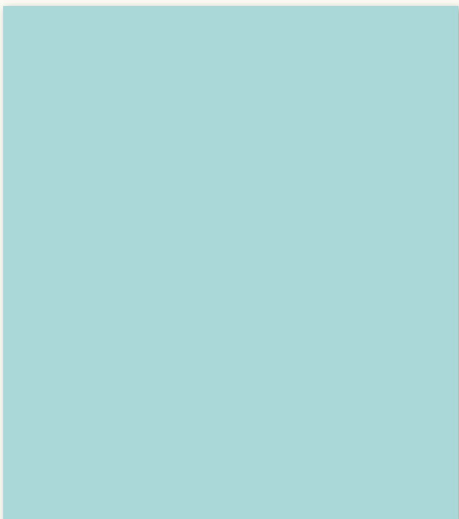
Regular Expressions
Pattern Matching

Topics

What is Pattern Matching?

Functions

Regular Expression



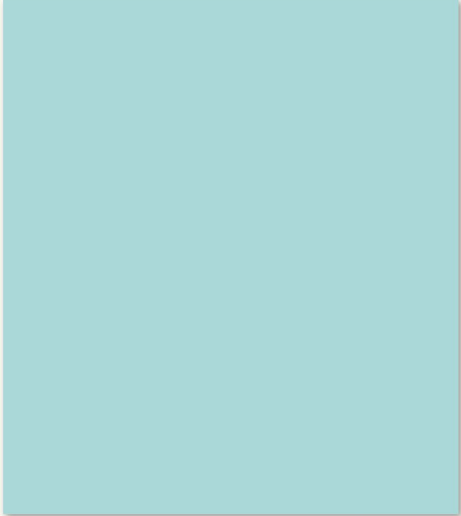
Regular Expressions
Pattern Matching

Topics

What is Pattern Matching?

Functions

Regular Expression
(Pattern)



Regular Expressions

Pattern Matching

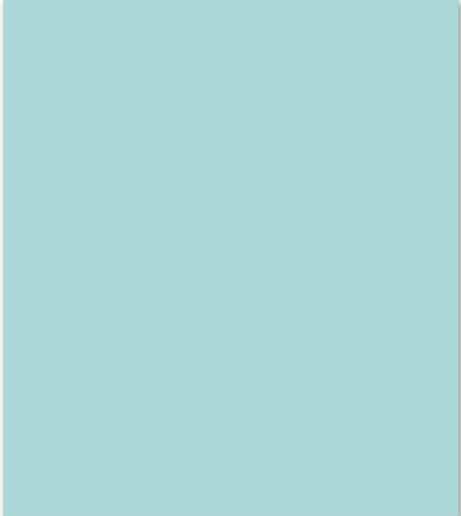
Topics

What is Pattern Matching?

Functions

Regular Expression
(Pattern)

- Quantifiers



Regular Expressions

Pattern Matching

Topics

What is Pattern Matching?

Functions

Regular Expression
(Pattern)

- Quantifiers
- Special Characters

Regular Expressions

Pattern Matching

Topics

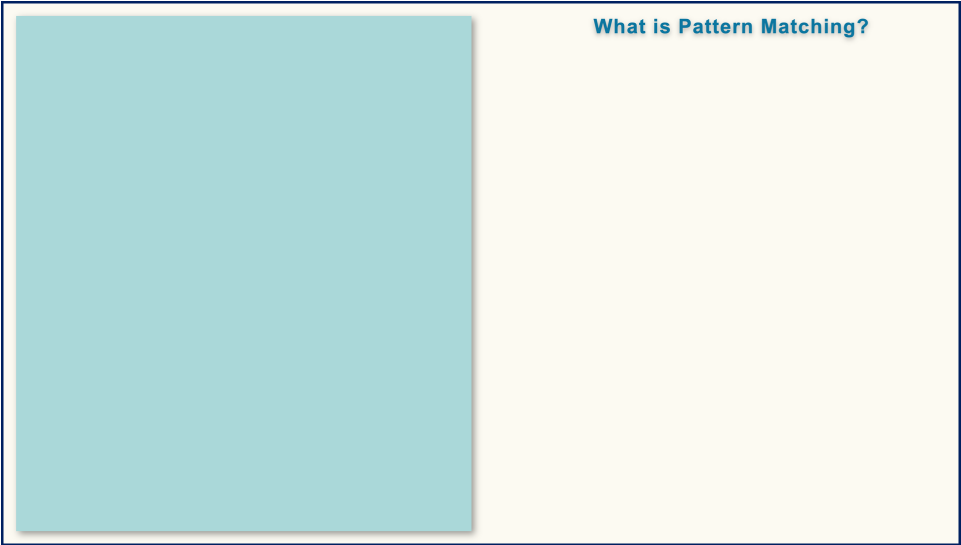
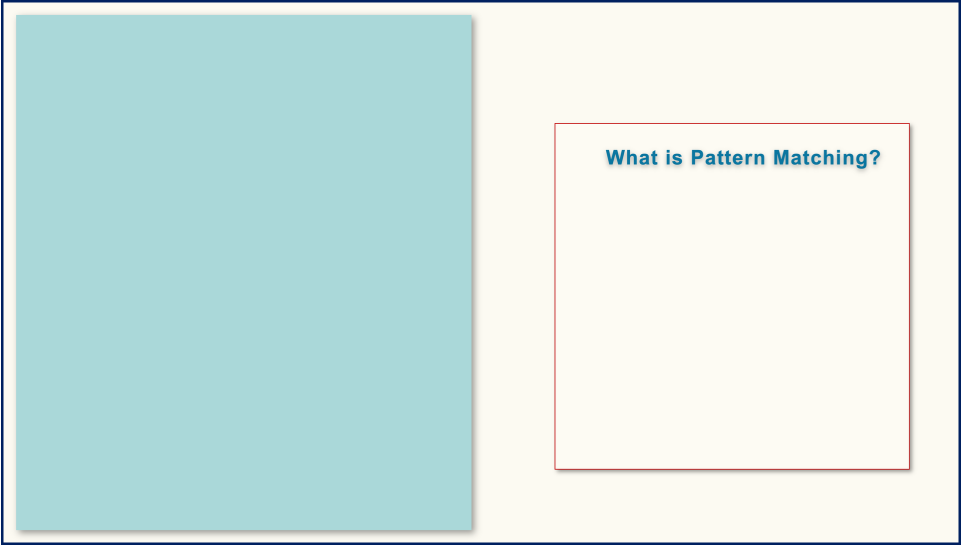
What is Pattern Matching?

Functions

Regular Expression

(Pattern)

- Quantifiers
- Special Characters
- Sequences



What is Pattern Matching?

pattern

'and'

What is Pattern Matching?

pattern

'and'

s1 = 'android'

What is Pattern Matching?

pattern

'and'

s1 = **'android'**

s2 = **'handle'**

What is Pattern Matching?

pattern

'and'

s1 = **'android'**

s2 = **'handle'**

What is Pattern Matching?

pattern

'and'

s1 = **'android'**

s2 = **'handle'**

s3 = **'and'**

What is Pattern Matching?

pattern

'and'

s1 = **'android'**

s2 = **'handle'**

s3 = **'and'**

What is Pattern Matching?

pattern

'com'

What is Pattern Matching?

pattern

'com|'

What is Pattern Matching?

pattern

'com|org'

What is Pattern Matching?

pattern

'com|org'

s1 = 'com'

What is Pattern Matching?

pattern

'com|org'

s1 = 'com'

s2 = 'org'

What is Pattern Matching?

pattern

'[abcd]'

What is Pattern Matching?

pattern

'[abcd]'

'a'

What is Pattern Matching?

pattern

'[abcd]'

'a'

'b'

What is Pattern Matching?

pattern

'[abcd]'

'a'

'b'

'c'

What is Pattern Matching?

pattern

'[abcd]'

'a' **'b'** **'c'** **'d'**

What is Pattern Matching?

pattern

'[abc]'

What is Pattern Matching?

pattern

'[abc]+'

What is Pattern Matching?

pattern

'[abc]+'

'a'

What is Pattern Matching?

pattern

'[abc]+'

'a'

'aa'

What is Pattern Matching?

pattern

'[abc]+'

'a'

'aa'

'aaaaaabbcc'

What is Pattern Matching?

pattern

'[abc]+'

'a'

'aa'

'aaaaaabbcc'

'b'

What is Pattern Matching?

pattern

'[abc]+'

'a'

'aa'

'aaaaaabbcc'

'b'

'bbb'

What is Pattern Matching?

pattern

'[abc]+'

'a'

'aa'

'aaaaaabbcc'

'b'

'bbb'

'babbcc'

What is Pattern Matching?

pattern

'[abc]+'

'a'

'aa'

'aaaaaabbcc'

'b'

'bbb'

'babbcc'

'c'

What is Pattern Matching?

pattern

'[abc]+'

'a'

'aa'

'aaaaaabbcc'

'b'

'bbb'

'babbcc'

'c'

'abcc'

What is Pattern Matching?

pattern

'[abc]+'

'a'

'aa'

'aaaaaabbcc'

'b'

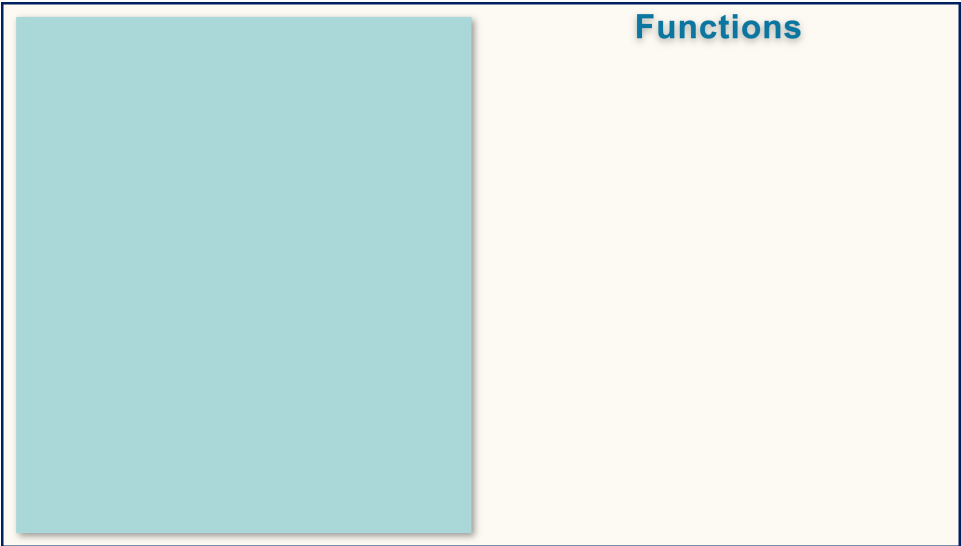
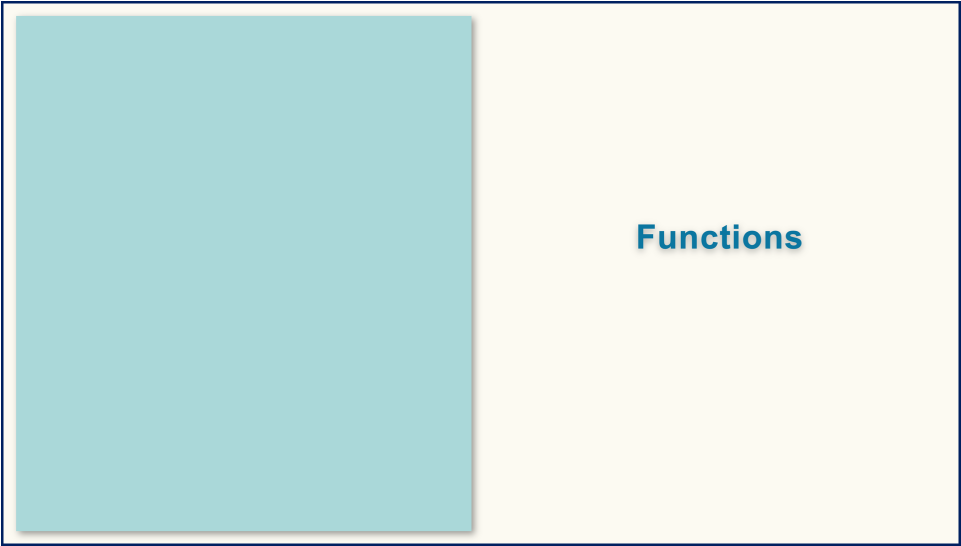
'bbb'

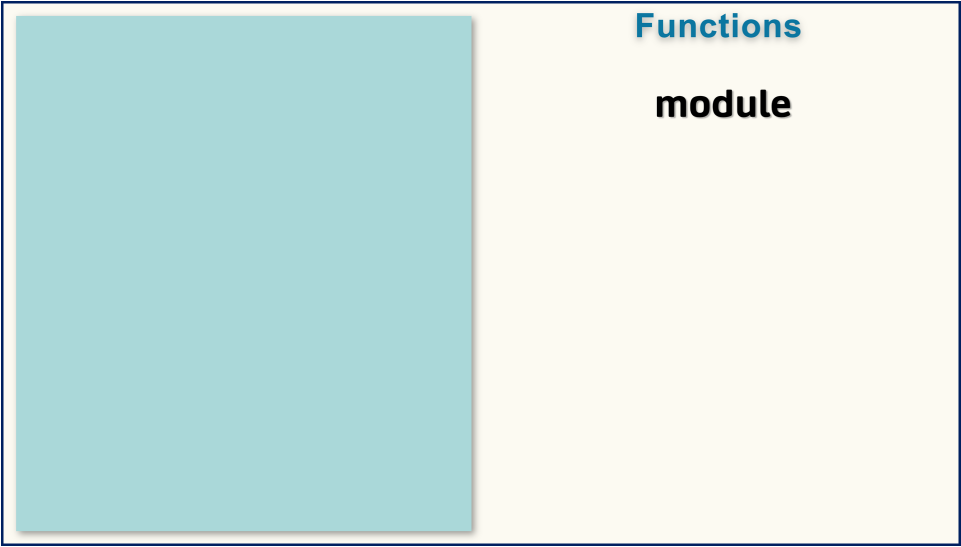
'babbcc'

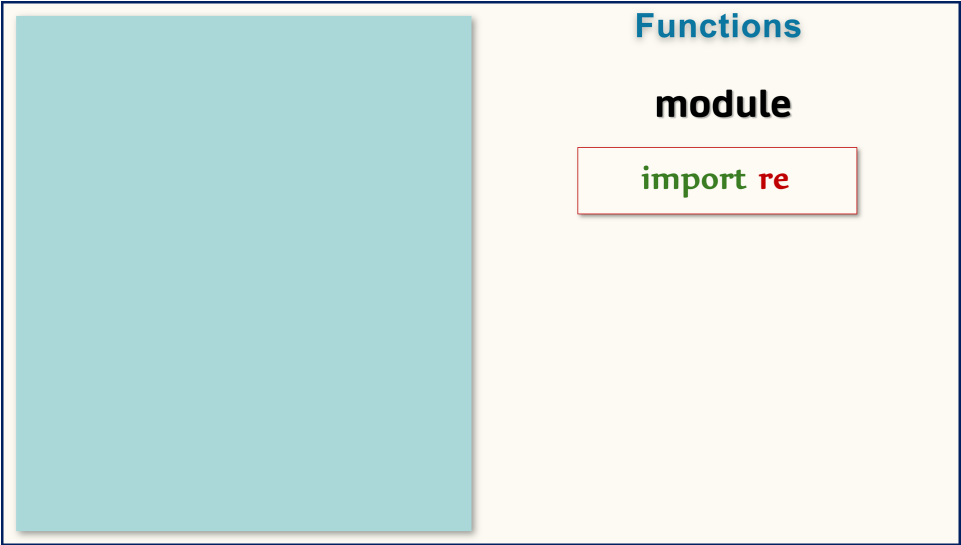
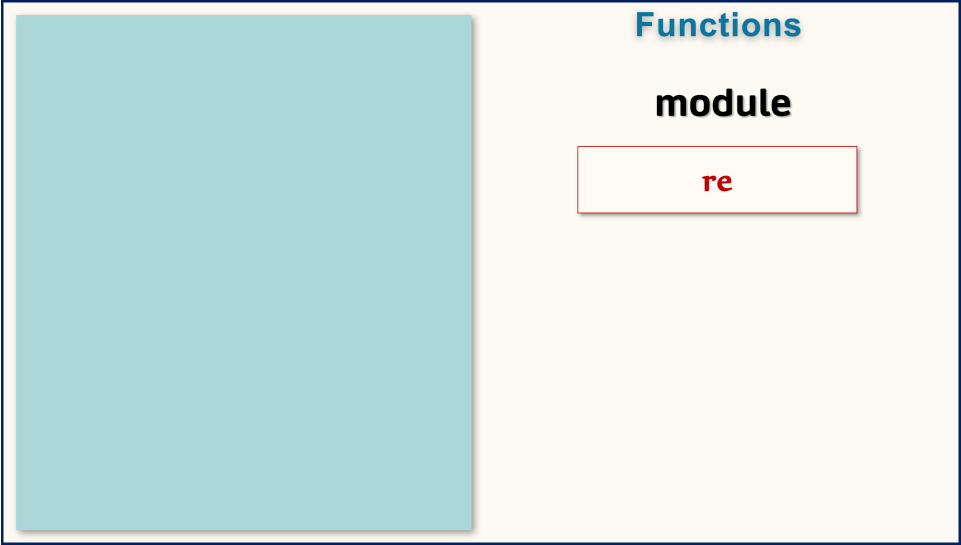
'c'

'abcc'

'ccbabb'







Functions

module

```
import re
```

```
import re
```

Functions

module

```
import re
```

```
import re as
```

Functions

module

```
import re
```

```
import re as regex
```

Functions

module

```
import re
```

```
import re as regex
```

```
from re
```

Functions

module

```
import re
```

```
import re as regex
```

```
from re import
```

Functions

```
compile( pattern, flag=0)
```

Functions

`compile( pattern, flag=0)`

Validation

Functions

`compile( pattern, flag=0)`

Validation

`match( pattern, string, flag=0)`

Functions

compile(pattern, flag=0)

Validation

match(pattern, string, flag=0)

fullmatch(pattern, string, flag=0)

Functions

compile(pattern, flag=0)

Validation

match(pattern, string, flag=0)

fullmatch(pattern, string, flag=0)

Search and Extract

Functions

compile(pattern, flag=0)

Validation

match(pattern, string, flag=0)

fullmatch(pattern, string, flag=0)

Search and Extract

search(pattern, string, flag=0)

Functions

compile(pattern, flag=0)

Validation

match(pattern, string, flag=0)

fullmatch(pattern, string, flag=0)

Search and Extract

search(pattern, string, flag=0)

findall(pattern, string, flag=0)

Functions

compile(pattern, flag=0)

Validation

match(pattern, string, flag=0)

fullmatch(pattern, string, flag=0)

Search and Extract

search(pattern, string, flag=0)

findall(pattern, string, flag=0)

split(pattern, string, maxsplit=0, flag=0)

